Design Documentation

Interactive House Project (Device Subgroup)

Revision History

Date	Version	Description	Author
2026-02-19	1.0	Initial design document	Patrick Nordahl
2026-03-10	1.1.2	Minor terminology refinement for clarity and consistency	Patrick Nordahl

Design item List

Requirement Name	Priority
D1. High-Level Architecture (Device → Gateway → Server → UI)	Essential
D2. Arduino Device Component Design (Controller, State, Sensors, Actuators)	Essential
D3. Serial Communication Protocol (Message types + structured format)	Essential
D4. Interaction Flow (Sequence: UI Command → Device Actuation → ACK/State Update)	Essential
D5. Fault Handling & Safe State Behavior	Essential

(Communication failure strategy)	

Design Item Descriptions

D1 High Level Architecture (Device -> Gateway -> Server -> UI)

Description:

The system follows a layered architecture where devices do not communicate directly with user interfaces. Instead, communication is routed through via a gateway (middleman) component, which maintains communication with the server.

The architecture is divided into four logical layers:

1. Device Layer (Arduino) – Handles sensor readings, sensor control, and maintains local device state.
2. Gateway Layer (Laptop/Raspberry Pi) – Reads and writes serial messages, validates data and forwards observations to/from the server.
3. Server Layer – Maintains system state and distributes commands to connected clients.
4. User Interface Layer – Allows users to observe and control device states.

The system distinguishes between requested state (from the server) and current state (on the device), ensuring consistency and fault tolerance.

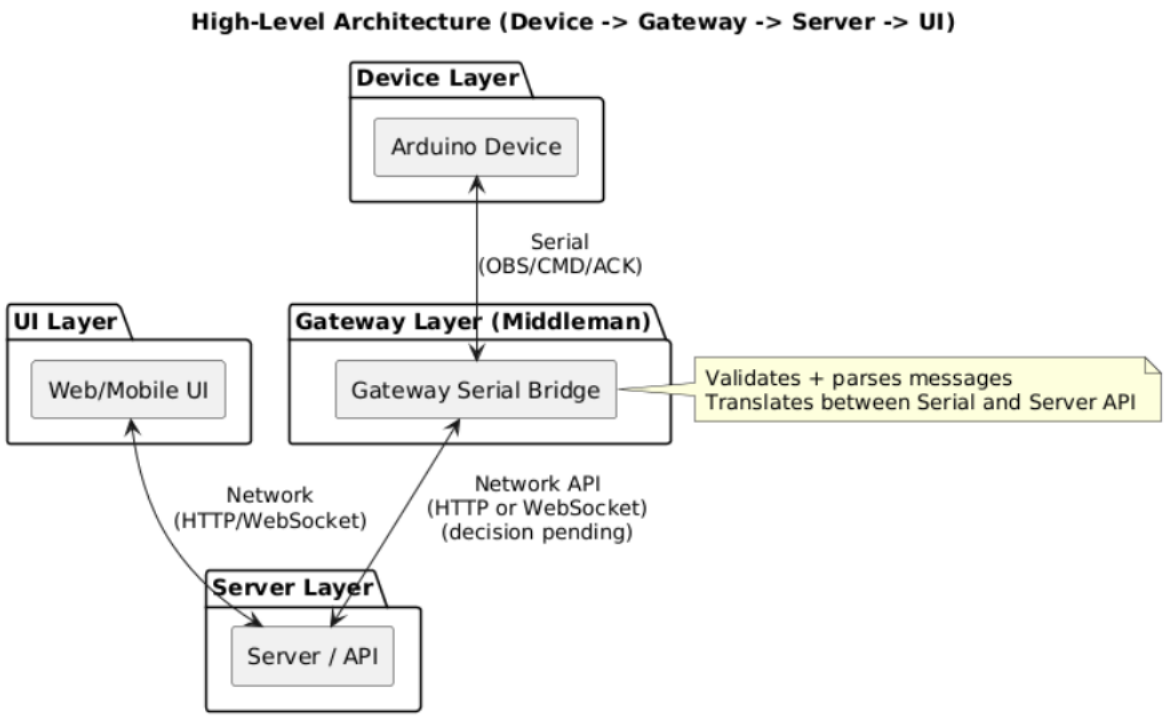

Figure 1. Architecture/Component Diagram (D1)

This design ensures separation of concerns and supports interoperability through the middleman architecture.

D2 Arduino Device Component Design

Description:

The Arduino device is structured into logical components to maintain modularity and separation of responsibilities.

The following conceptual components are defined:

- DeviceController – Main coordination loop (loop execution, responsible for updates).
- SensorManager – Reads sensors values and normalizes data.
- ActuatorManager – Applies state changes to physical hardware.
- DeviceState / HouseState – Stores the current device state.
- SerialProtocol Module – Encodes outgoing messages and decodes incoming commands.

This modular structure improves maintainability and reduces coupling between sensors, state management and communication.

D3. Serial Communication Protocol

Description:
Structured Serial communication is used between the Arduino device and the gateway to ensure reliable data exchange.

Three conceptual message types are defined:

- OBS (Observation) – Sent from device to gateway (sensor values + state).
- CMD (Command) – Sent from gateway to device (state change request).
- ACK (Acknowledgement) – Sent from device to confirm applied command.

Example key-value format:

- obs;gas=312;motion=1;light=0
- cmd;light=1
- ack;light=1

Final message format (JSON or key-value) remains an open design decision.
Structured messaging reduces integration risks and supports validation.

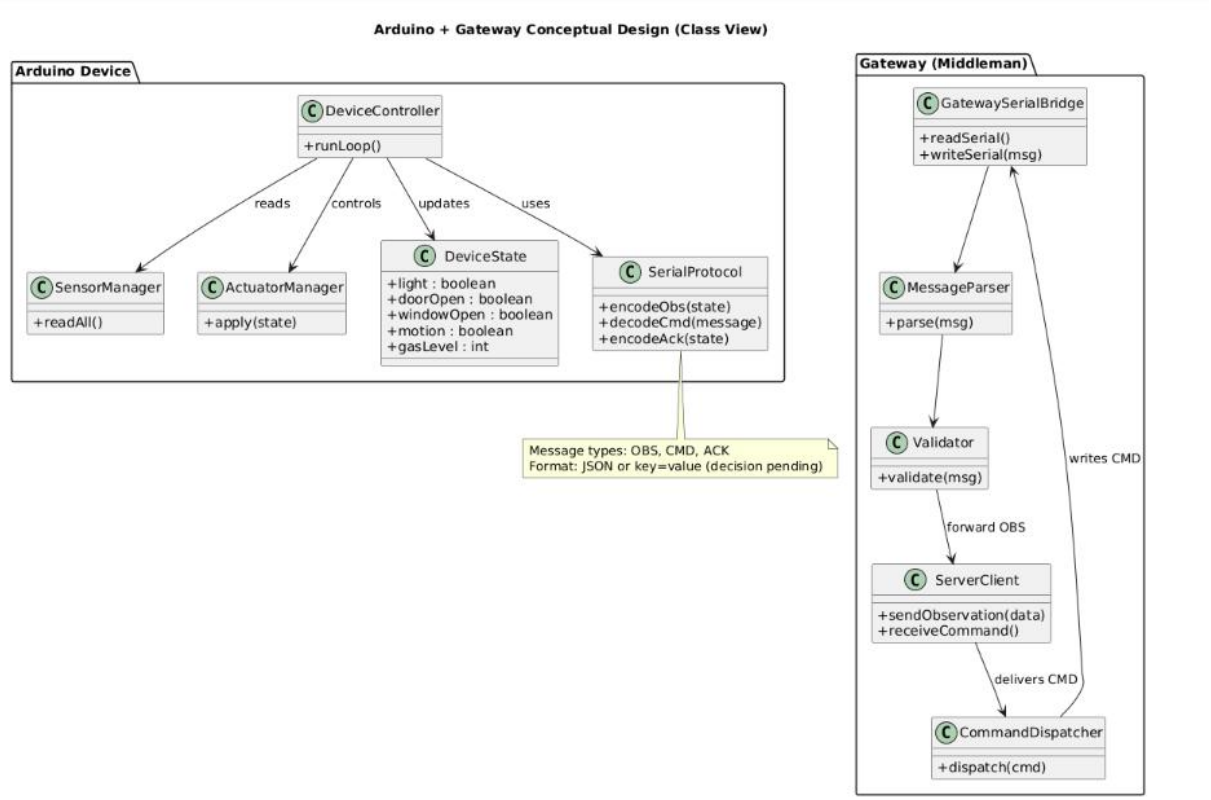

Figure 2. Class diagram (D2+D3)

D4 Interaction Flow – UI Command to Device

Description:

The following interactions sequence describes how a user command goes through the system.

Scenario: User toggles light from UI

1. UI sends command to server.
2. Server forwards command to gateway.
3. Gateway writes CMD message via serial
4. Arduino decodes CMD message.
5. DeviceController updates DeviceState.
6. ActuatorManager applies physical change.
7. Arduino sends ACK + updated state back via Serial.

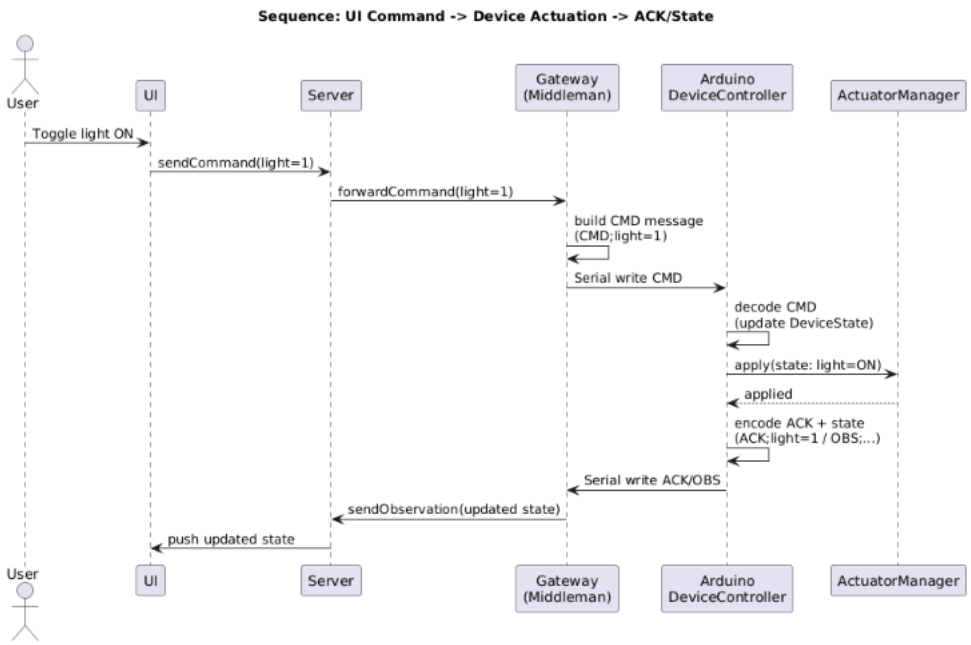

Figure 3. Sequence Diagram (D4)

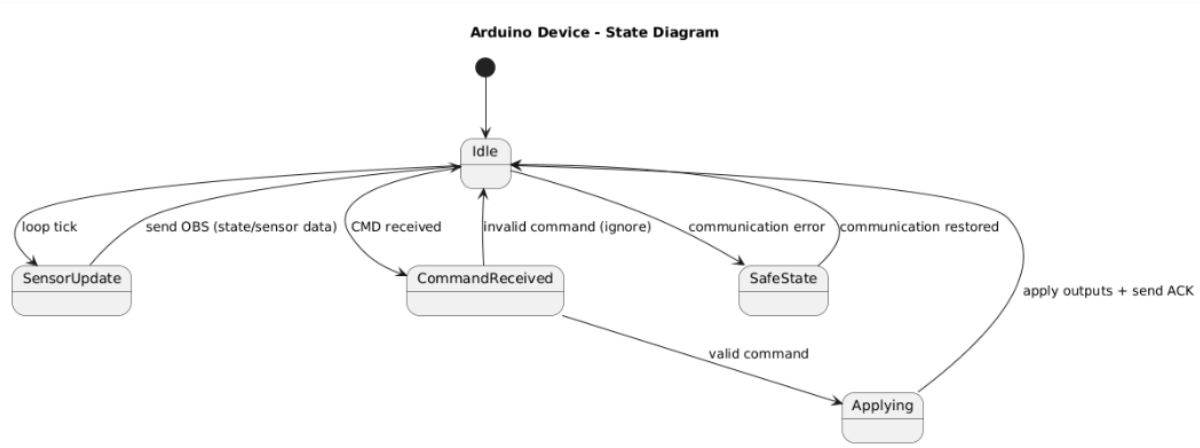

Figure 4. State Diagram (Arduino Device)

This flow ensures end-to-end synchronization between UI, server and device.

D5 Fault Handling & Safe Behaviour

Description:

To improve reliability, the device must handle communication failures gracefully.

If Serial communication is interrupted or corrupted:

- The Arduino continues operating in its last known safe state.
- Incoming commands must be validated before actuation.
- Invalid messages is ignored.
- The gateway is responsible for dropping corrupted data and logging errors.

This design reduces the impact of communication instability and addresses identified integration and reliability risks.